



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

Робототехника и комплексная автоматизация (РК)

КАФЕДРА

Системы автоматизированного проектирования (РК6)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОМУ ПРОЕКТУ

НА ТЕМУ:

«Разработка сайта для проведения обучающих курсов»

Студент РК6-31М

(Подпись, дата)

Онюшев А.А.

И.О. Фамилия

Руководитель курсового проекта

(Подпись, дата)

Витюков Ф.А.

И.О. Фамилия

2026 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой РК6
А.П. Карпенко

« ____ » _____ 20 ____ г.

ЗАДАНИЕ на выполнение курсового проекта

по дисциплине _____

Студент группы РК6-31М

Онюшев Артем Андреевич
(Фамилия, имя, отчество)

Тема курсового проекта: Разработка сайта для проведения обучающих курсов

Направленность КП (учебный, исследовательский, практический, производственный, др.) учебный
Источник тематики (кафедра, предприятие, НИР) кафедра

График выполнения проекта: 25% к 5 нед., 50% к 11 нед., 75% к 14 нед., 100% к 16 нед.

Задание. Исследовать существующие системы аутентификации и авторизации. Реализовать собственную систему SSO используя ORY Keto. Спроектировать архитектуру сайта для проведения обучающих курсов. Разработать пользовательский интерфейс в Figma

Оформление курсового проекта:

Расчетно-пояснительная записка на 27 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.):

Дата выдачи задания «8» октября 2025 г.

Руководитель курсовой работы

(Подпись, дата)

Витюков Ф.А.

И.О. Фамилия

Студент

(Подпись, дата)

Онюшев А.А.

И.О. Фамилия

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

АННОТАЦИЯ

Курсовая работа посвящена разработке сайта для проведения обучающих курсов с использованием языка программирования Golang. В работе рассматриваются вопросы проектирования архитектуры образовательной веб-платформы с акцентом на выбор и обоснование системы аутентификации и авторизации пользователей. Проведен аналитический обзор существующих решений в области Single Sign-On (SSO), включая готовые экосистемы и вариант разработки собственной системы. На основе сравнительного анализа обоснован выбор экосистемы ORY, включающей сервисы Hydra и Keto.

В рамках работы также рассмотрены вопросы выбора технологий хранения данных и инфраструктуры размещения. Обосновано использование реляционной базы данных PostgreSQL для хранения основной информации, in-memory хранилища Redis для работы с токенами и сессиями, а также объектного S3-хранилища для размещения видеоматериалов и учебных ресурсов. Проанализированы требования к хостинг-платформе и обоснован выбор провайдера Selectel.

В расчётно-пояснительной записке 27 страниц, 18 рисунок.

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	3
ОСНОВНЫЕ ПОНЯТИЯ И ОПРЕДЕЛЕНИЯ	5
ВВЕДЕНИЕ	6
1. Системы аутентификации и авторизации.....	8
1.1. Обзор существующих SSO-решений	9
1.2. Получившаяся система.....	14
2. Проектирование архитектуры	17
1. Выбор языка программирования	19
2. Взаимодействие сервисов системы	20
3. Технологии хранения данных	23
1. Реляционная база данных	24
2. In-memory база данных	25
3. Объектное хранилище.....	26
4. Пользовательский интерфейс	27
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	30

ОСНОВНЫЕ ПОНЯТИЯ И ОПРЕДЕЛЕНИЯ

SSO – (Single Sign On) Система единого входа.

СУБД – Система Управления Базами Данных.

PostgreSQL – Одна из известных СУБД.

Golang – Язык программирования.

Kubernetes – Платформа оркестрации контейнеризованных приложений.

Под – Группа из одного или нескольких контейнеров с общими хранилищами и сетевыми ресурсами, а также спецификация для запуска контейнеров.

ВВЕДЕНИЕ

В последние годы наблюдается рост интереса к онлайн-обучению и дистанционным образовательным технологиям, охватывающим формальное образование, корпоративное обучение и повышение квалификации. В условиях высокой конкуренции между образовательными платформами особое значение приобретает качество программной реализации обучающих систем, а также их надежность, безопасность и масштабируемость.

Современные сайты для проведения обучающих курсов представляют собой сложные программные системы, включающие механизмы управления пользователями, хранения и доставки учебных материалов и контроля доступа к контенту. Одной из ключевых задач при их разработке является обеспечение корректной и безопасной аутентификации и авторизации пользователей, поскольку ошибки на данном уровне могут приводить к утечке данных и нарушению образовательного процесса.

При проектировании образовательных платформ особое внимание уделяется реализации механизмов единого входа (Single Sign-On, SSO), позволяющих пользователю получать доступ ко всем компонентам системы после однократной аутентификации. Выбор архитектуры SSO требует тщательного анализа, так как такие решения напрямую влияют на безопасность, производительность и масштабируемость системы.

На этапе проектирования возникает необходимость выбора между использованием готовых решений для аутентификации и авторизации и разработкой собственной системы. Готовые решения сокращают сроки разработки, однако могут накладывать архитектурные ограничения и приводить к зависимости от внешних сервисов, тогда как собственная реализация требует значительных трудозатрат и высокой экспертизы в области информационной безопасности.

Помимо системы аутентификации, при разработке обучающей платформы важным является обоснованный выбор технологий хранения данных и

инфраструктуры размещения. Образовательные системы работают с пользовательскими данными, информацией о курсах и мультимедийным контентом, что предъявляет повышенные требования к надежности и масштабируемости.

Целью курсовой работы является разработка сайта для проведения обучающих курсов с использованием языка программирования Golang и обоснование выбора ключевых архитектурных и технологических решений, включая систему аутентификации и авторизации, базы данных и платформу хостинга.

Для достижения поставленной цели в работе предполагается решение следующих задач:

- проанализировать особенности и требования к системам аутентификации и авторизации в образовательных веб-платформах;
- исследовать существующие SSO-решения и подходы к их реализации;
- рассмотреть возможность разработки собственной SSO-системы и оценить ее целесообразность;
- спроектировать архитектуру сайта для проведения обучающих курсов;
- выбрать и обосновать технологии хранения данных, включая реляционную базу данных, in-memory хранилище и объектное хранилище;
- обосновать выбор платформы хостинга для размещения разрабатываемой системы;
- описать основные пользовательские сценарии и элементы пользовательского интерфейса.

1. Системы аутентификации и авторизации

Разработка сайта для проведения обучающих курсов требует особого внимания к вопросам аутентификации и авторизации пользователей. Данные механизмы являются основой безопасности веб-приложения и напрямую влияют на устойчивость системы, корректность разграничения доступа и защиту персональных данных. В образовательных платформах данные вопросы приобретают дополнительную значимость, поскольку система обслуживает большое количество пользователей с различными ролями и уровнями доступа.

Аутентификация представляет собой процесс проверки подлинности пользователя, то есть подтверждение того, что пользователь действительно является тем, за кого себя выдает. Авторизация, в свою очередь, определяет, какие действия и ресурсы доступны аутентифицированному пользователю. В контексте обучающих платформ это может включать доступ к курсам, учебным материалам, административным функциям, результатам обучения и другим компонентам системы.

В современных веб-приложениях аутентификация и авторизация редко реализуются в виде изолированного модуля. Как правило, они являются частью распределённой архитектуры и взаимодействуют с другими сервисами, что требует стандартизированных протоколов и хорошо продуманной архитектуры.

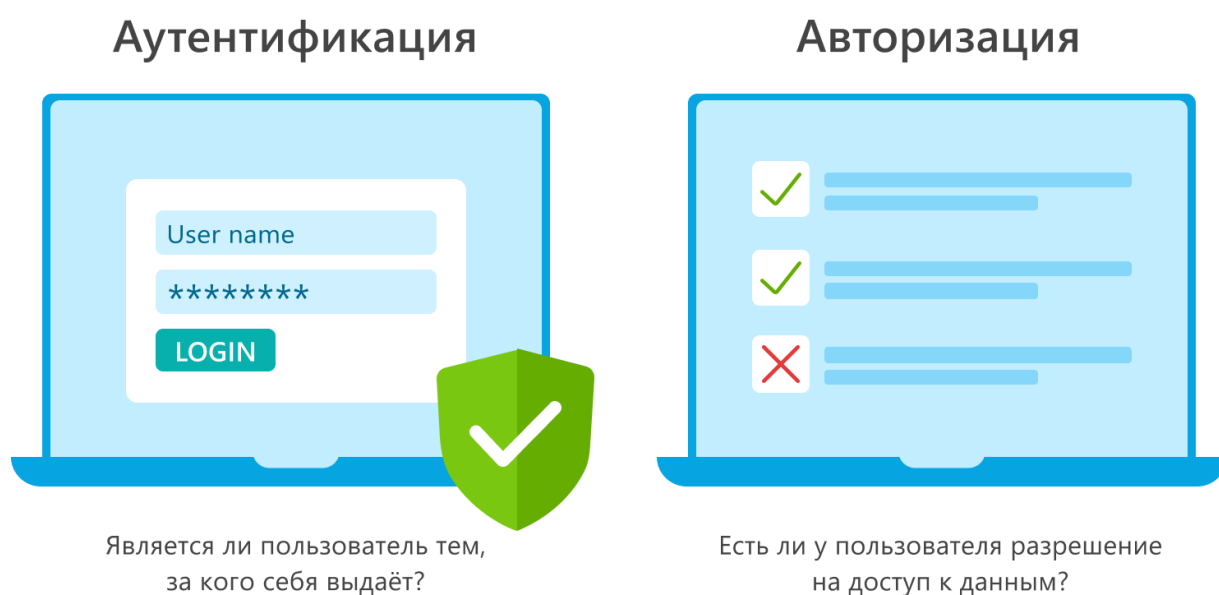


Рис. 1 Аутентификация и авторизация

1.1. Обзор существующих SSO-решений

Одним из распространённых подходов к организации аутентификации в сложных веб-системах является использование механизма единого входа (Single Sign-On, SSO). SSO позволяет пользователю пройти процедуру аутентификации один раз и затем получать доступ ко всем связанным сервисам без повторного ввода учетных данных.

В образовательных платформах использование SSO особенно актуально, поскольку такие системы часто состоят из нескольких компонентов, включая пользовательский интерфейс, административную панель, сервисы хранения данных, сервисы аналитики и интеграции с внешними системами. Повторная аутентификация при каждом обращении к отдельному компоненту снижает удобство использования и увеличивает вероятность ошибок.

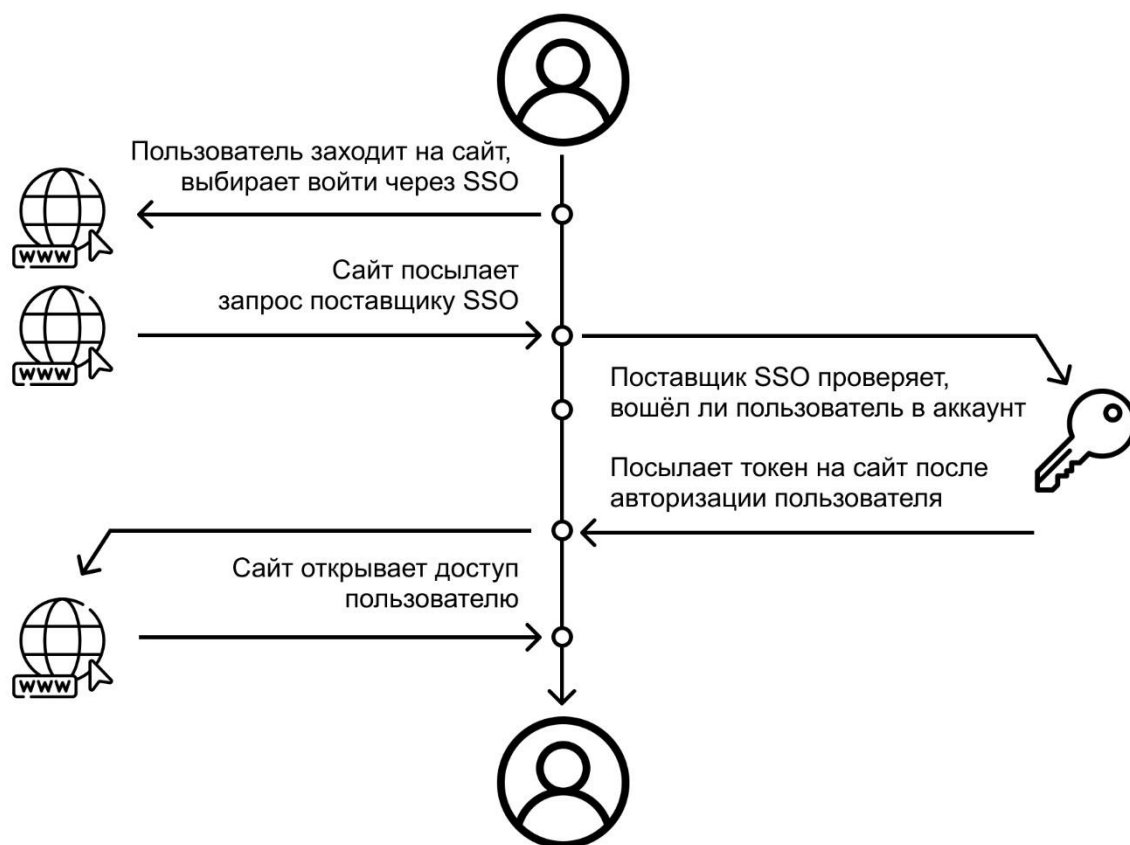


Рис. 2 схема работы SSO

На современном рынке представлено множество решений для реализации аутентификации и авторизации с поддержкой SSO. Эти решения различаются по архитектуре, сложности внедрения, возможностям настройки и модели

распространения. В рамках данной работы рассмотрены наиболее распространённые подходы и системы.

Одним из базовых стандартов в области аутентификации является протокол OAuth 2.0, который широко используется для делегирования доступа между сервисами. OAuth 2.0 определяет набор потоков авторизации, позволяющих приложениям получать доступ к защищённым ресурсам от имени пользователя. На основе OAuth 2.0 был разработан протокол OpenID Connect, который расширяет его возможности и добавляет механизм идентификации пользователя.

Использование OAuth 2.0 и OpenID Connect позволяет реализовать безопасную аутентификацию без передачи учетных данных сторонним сервисам. Однако данные протоколы представляют собой спецификации и требуют дополнительной реализации серверной части, что усложняет разработку при отсутствии готовой инфраструктуры.

Одним из популярных готовых решений является система Keycloak, представляющая собой сервер аутентификации с поддержкой OAuth 2.0, OpenID Connect и SAML. Keycloak предоставляет широкий функционал, включая управление пользователями, ролями и политиками доступа. В то же время данное решение обладает монолитной архитектурой и требует значительных ресурсов при масштабировании, что может быть критично для высоконагруженных систем.

Коммерческие облачные решения, такие как Auth0, предлагают готовую инфраструктуру для аутентификации и авторизации с минимальными затратами на внедрение. Основным недостатком таких решений является высокая стоимость при росте количества пользователей и зависимость от внешнего провайдера, что может ограничивать гибкость архитектуры и усложнять соблюдение требований по локализации данных.

Существуют также более простые решения, такие как Firebase Authentication, ориентированные на быстрое создание приложений. Однако подобные системы плохо подходят для сложных сценариев разграничения

доступа и микросервисных архитектур, характерных для образовательных платформ.



Рис. 3 Логотипы готовых SSO решений.

Альтернативой использованию готовых решений является разработка собственной системы аутентификации и авторизации. Данный подход позволяет полностью контролировать архитектуру и логику работы SSO, а также адаптировать систему под конкретные требования проекта.

При этом разработка собственной SSO-системы связана с рядом существенных сложностей. В первую очередь это вопросы безопасности, включая защиту учетных данных, корректную реализацию криптографических алгоритмов, защиту от атак повторного воспроизведения и утечек токенов. Ошибки в реализации могут привести к уязвимостям, устранение которых потребует значительных усилий.

Кроме того, собственная SSO-система требует постоянного сопровождения, обновления и аудита безопасности. При росте нагрузки возникает необходимость в оптимизации и масштабировании, что дополнительно увеличивает сложность поддержки. С учётом данных факторов разработка собственной системы аутентификации оправдана, как правило, только в крупных проектах с особыми требованиями.

Для выбора оптимального решения по реализации аутентификации и авторизации в разрабатываемой обучающей платформе был проведён сравнительный анализ различных подходов и существующих SSO-решений. Целью анализа являлось выявление решения, наиболее полно удовлетворяющего требованиям безопасности, масштабируемости, гибкости настройки и экономической целесообразности.

Решение SSO	Безопасность	Масштабируемость	Гибкость настройки	Внедрение	Поддержка	Стоимость
Keycloak	4	3	3	3	4	2
Auth0	5	5	3	5	5	4
Собственная	2	2	5	1	1	5
ORY	4	5	5	4	4	1

Рис. 4 Сравнительный анализ

Анализ таблицы показывает, что монолитные решения, такие как Keycloak, обеспечивают приемлемый уровень безопасности, однако обладают ограниченной гибкостью и требуют значительных ресурсов при масштабировании. Облачные сервисы, такие как Auth0, демонстрируют высокий уровень безопасности и удобство внедрения, но характеризуются высокой стоимостью владения и зависимостью от внешнего провайдера.

Разработка собственной SSO-системы позволяет добиться максимальной гибкости, однако сопровождается высокими рисками, связанными с безопасностью, а также значительными трудозатратами на разработку и сопровождение. В условиях ограниченных ресурсов данный подход является экономически нецелесообразным.

Экосистема ORY демонстрирует сбалансированные показатели по всем ключевым критериям. Модульная архитектура, соответствие стандартам OAuth 2.0 и OpenID Connect, а также ориентация на облачные и микросервисные среды делают данное решение оптимальным для использования в образовательной платформе. При этом сама система имеет открытый код, написанный на Golang.

По результатам проведённого анализа было принято решение использовать экосистему ORY для реализации аутентификации и авторизации. Данная экосистема включает специализированные сервисы, каждый из которых отвечает за отдельную функциональную область.

ORY Hydra используется для реализации OAuth 2.0 и OpenID Connect и отвечает за выпуск и валидацию токенов доступа. ORY Keto применяется для управления политиками доступа и реализации моделей разграничения прав. Такое разделение ответственности позволяет построить гибкую и

масштабируемую систему аутентификации, соответствующую требованиям микросервисной архитектуры.

Таким образом, выбор экосистемы ORY является обоснованным с точки зрения безопасности, масштабируемости и соответствия архитектурным требованиям разрабатываемой обучающей платформы.

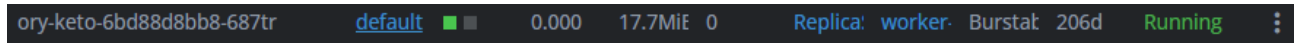


Рис. 5 Запущенный под Ory Keto в Kubernetes

1.2. Получившаяся система

Процесс взаимодействия пользователя с системой начинается со страницы входа, на которой предусмотрена возможность авторизации по номеру телефона или адресу электронной почты с использованием пароля. Интерфейс страницы разработан с учетом требований минимализма и удобства восприятия, что снижает вероятность ошибок при вводе данных и повышает общую usability системы. помогут лучше понять, как заполнение и длина шага работают.

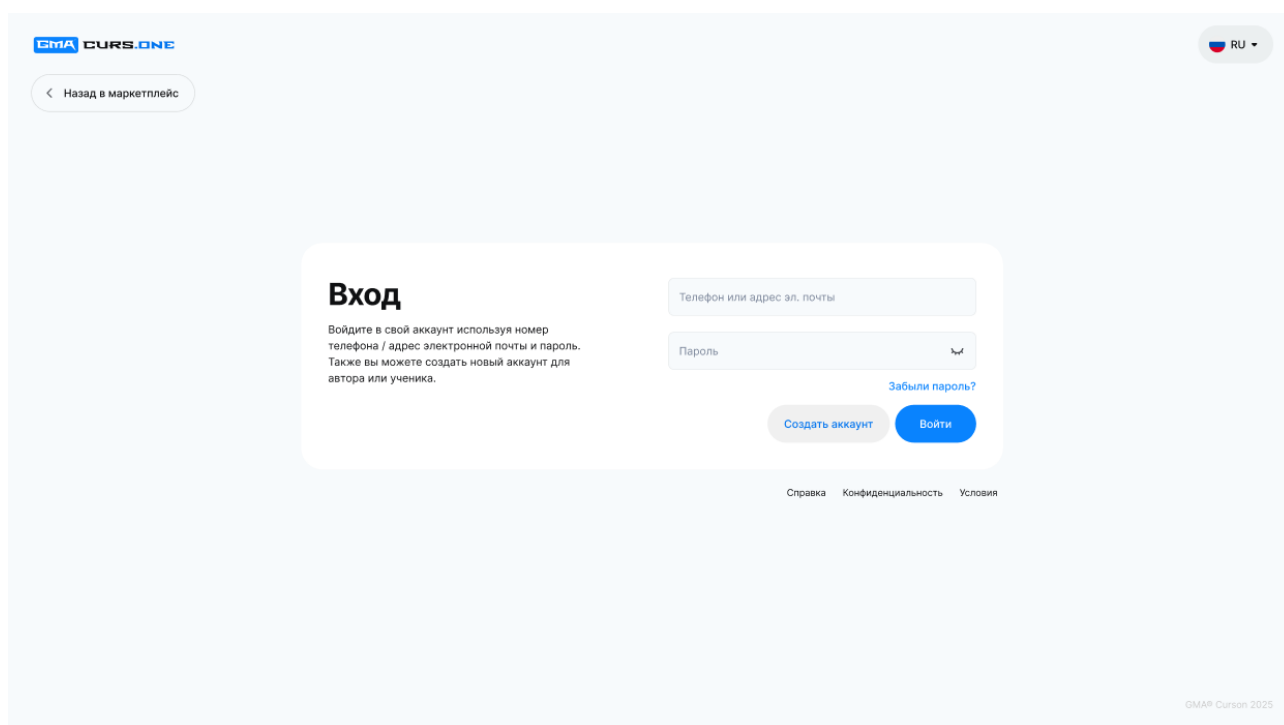


Рис. 6 Экран входа

В случае отсутствия учетной записи пользователь может перейти к процедуре регистрации. Учтена возможность регистрации аккаунта, как для автора, так и для ученика

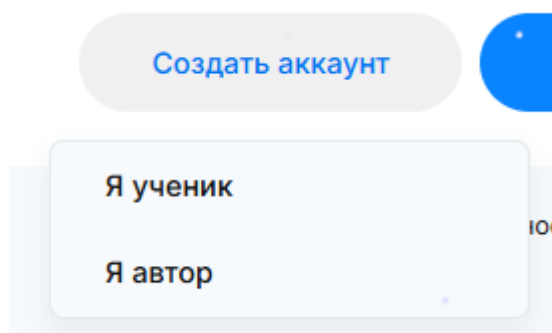


Рис. 7 Меню выбора роли пользователя

На первом этапе регистрации осуществляется ввод персональных данных пользователя, включая фамилию, имя и отчество (при необходимости). Данный шаг позволяет сформировать базовый профиль пользователя в системе и подготовить данные для дальнейшей идентификации.

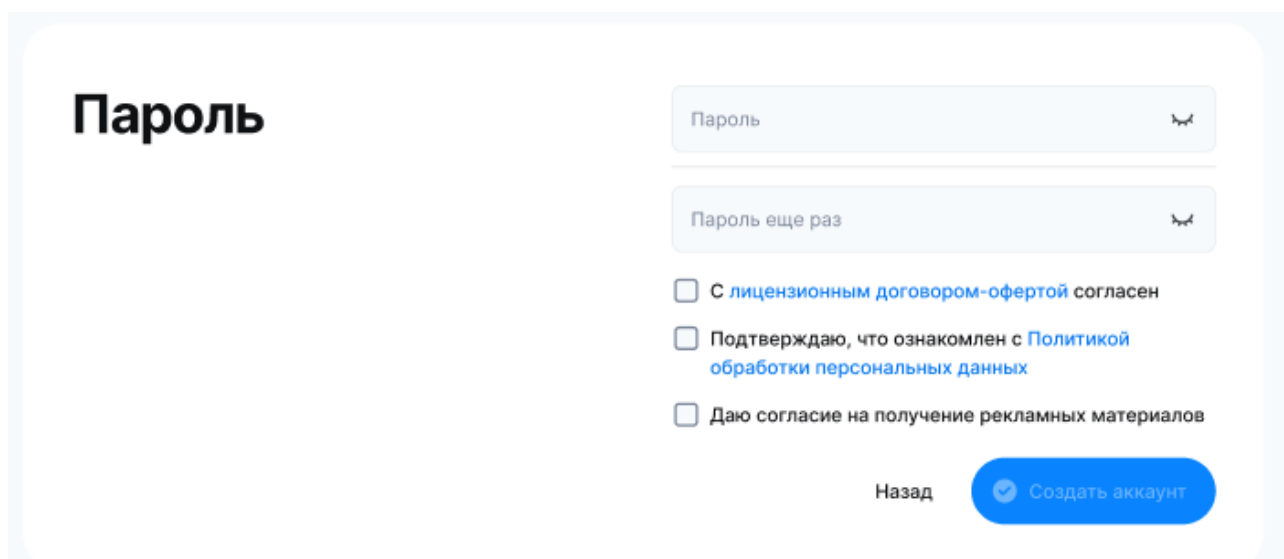
Рис. 8 Первый шаг регистрации ученика

Следующим этапом является подтверждение контактного номера телефона. Пользователь вводит номер телефона, после чего система инициирует отправку одноразового кода подтверждения. Использование механизма подтверждения номера телефона повышает достоверность учетных данных и снижает риск создания фиктивных учетных записей.

Рис. 9 Подтверждение номера телефона

После подтверждения контактных данных пользователю предлагается задать пароль для последующей аутентификации. На данном этапе также реализовано получение согласий на обработку персональных данных и условия

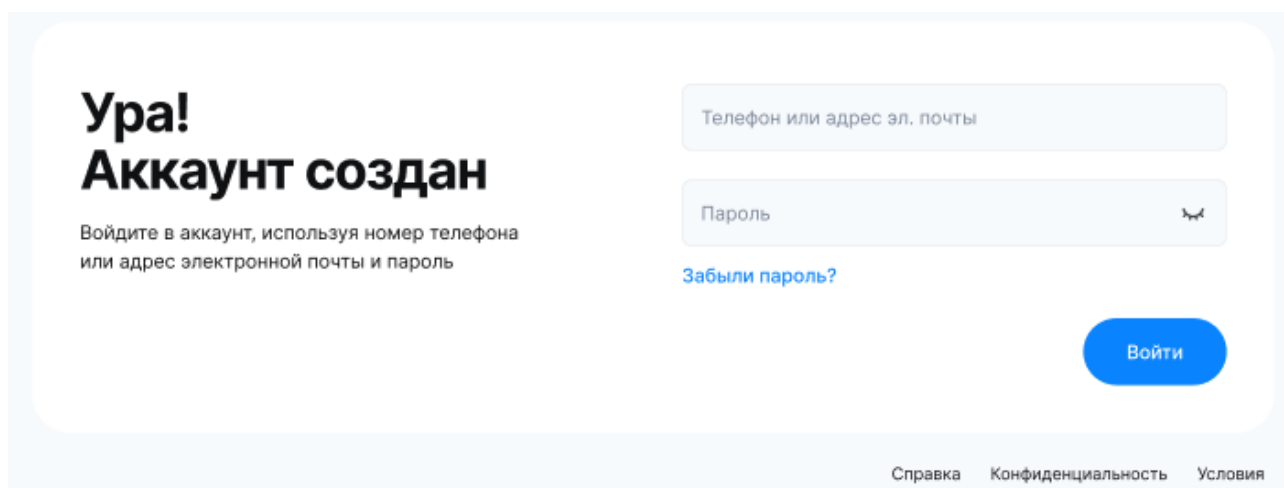
использования сервиса, что обеспечивает соответствие требованиям законодательства в области защиты информации.



The screenshot shows a registration form titled "Пароль" (Password). It contains two input fields: "Пароль" (Password) and "Пароль еще раз" (Password again). Below these fields are three checkboxes with labels: "С лицензионным договором-офертой согласен" (Agree with the license agreement-offer), "Подтверждаю, что ознакомлен с Политикой обработки персональных данных" (I confirm that I am familiar with the Policy on processing personal data), and "Даю согласие на получение рекламных материалов" (I give consent to receive advertising materials). At the bottom right, there is a "Назад" (Back) link and a blue button labeled "Создать аккаунт" (Create account).

Рис. 10 Подтверждение номера телефона

Завершающим этапом регистрации является создание учетной записи и переход пользователя к процедуре входа в систему. После успешной регистрации пользователь может выполнить аутентификацию с использованием указанных ранее учетных данных и получить доступ к функциональным возможностям образовательной платформы.



The screenshot shows a confirmation screen titled "Ура! Аккаунт создан" (Hooray! Account created). Below the title, it says "Войдите в аккаунт, используя номер телефона или адрес электронной почты и пароль" (Log in to the account using the phone number or email address and password). On the right side, there are two input fields: "Телефон или адрес эл. почты" (Phone or email address) and "Пароль" (Password). Below the password field is a link "Забыли пароль?" (Forgot password?). At the bottom right, there is a blue button labeled "Войти" (Log in). At the bottom of the page, there are links for "Справка" (Help), "Конфиденциальность" (Privacy), and "Условия" (Terms).

Рис. 11 Экран успешного создания аккаунта

2. Проектирование архитектуры

Проектирование архитектуры веб-платформы для проведения обучающих курсов является ключевым этапом разработки, определяющим надёжность, масштабируемость, безопасность и удобство сопровождения программной системы. На данном этапе формируются структурные решения, определяются основные компоненты системы, способы их взаимодействия, а также выбираются технологические подходы, обеспечивающие устойчивую работу платформы в условиях изменяющейся нагрузки.

Современные образовательные веб-системы функционируют в распределённой среде и должны обеспечивать одновременную работу большого числа пользователей, хранение значительных объёмов учебных данных и мультимедийного контента, а также безопасную обработку персональной информации. В связи с этим архитектура разрабатываемого сайта строится с учётом принципов модульности, масштабируемости и отказоустойчивости.

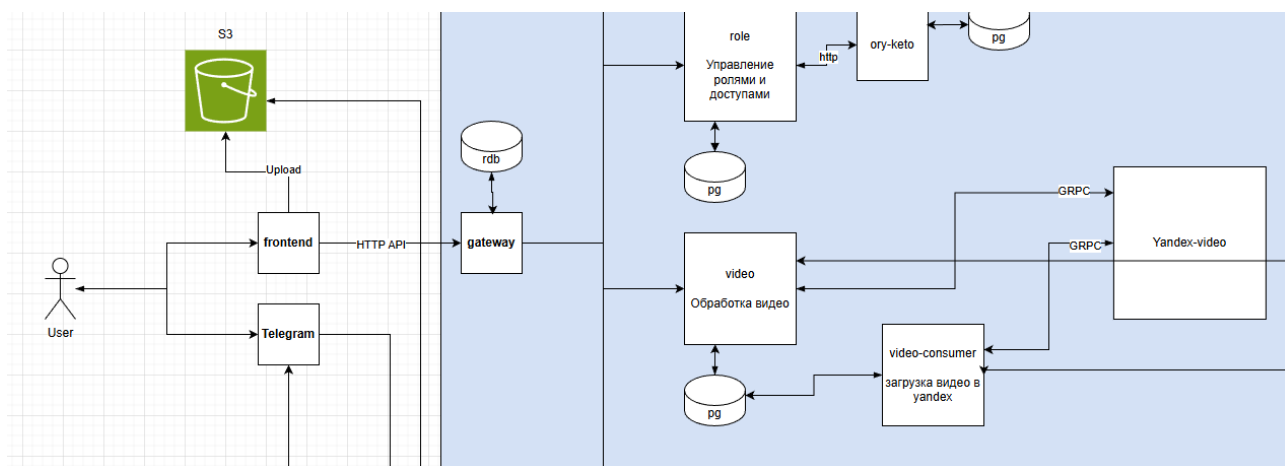


Рис. 12 Часть получившейся схемы архитектуры

Архитектура сайта для проведения обучающих курсов реализуется по многоуровневой модели, включающей клиентский уровень, серверный уровень обработки бизнес-логики, подсистему аутентификации и авторизации, систему хранения данных и инфраструктурные сервисы.

Клиентский уровень представлен веб-интерфейсом пользователя, обеспечивающим доступ к функциональности платформы: регистрации, авторизации, просмотру курсов, прохождению учебных модулей и

взаимодействию с образовательным контентом. Интерфейс проектируется с учётом принципов удобства использования, адаптивности и минимизации когнитивной нагрузки на пользователя.

Серверный уровень отвечает за реализацию бизнес-логики системы, обработку запросов клиентов, управление доступом к данным и взаимодействие с внешними сервисами. Данный уровень выступает центральным элементом архитектуры и обеспечивает согласованную работу всех компонентов платформы.

Подсистема аутентификации и авторизации вынесена в отдельный логический контур и реализуется с использованием специализированных сервисов SSO-архитектуры. Такое разделение повышает безопасность системы и упрощает масштабирование.

Система хранения данных включает реляционную базу данных для хранения структурированной информации, in-memory хранилище для ускорения операций аутентификации и объектное хранилище для размещения мультимедийных материалов.

1. Выбор языка программирования

При выборе технологического стека серверной части учитывались требования к производительности, отказоустойчивости, скорости разработки и удобству сопровождения. Для реализации backend-части был выбран язык программирования Golang.

Golang характеризуется высокой скоростью выполнения, встроенной поддержкой конкурентности, эффективным управлением памятью и простотой развертывания приложений. Эти особенности делают его оптимальным выбором для разработки высоконагруженных сетевых сервисов и микросервисных архитектур.

Дополнительным преимуществом является возможность компиляции статически связанного исполняемого файла, что упрощает процесс деплоя и снижает зависимость от окружения. Также язык обладает развитой стандартной библиотекой для работы с HTTP-соединениями, криптографией и параллельными вычислениями.

Использование Golang позволяет обеспечить стабильную работу серверной части платформы при росте числа пользователей и объема обрабатываемых данных.



Рис. 13 Логотип Golang

2. Взаимодействие сервисов системы

Взаимодействие компонентов разрабатываемой платформы организовано по микросервисной модели с выделением единой точки входа в виде основного сервиса gateway. Все внешние HTTP-запросы от клиентского приложения поступают в gateway, который выполняет функции маршрутизации, первичной валидации запросов и централизованного применения сквозных политик (аутентификация, авторизация, логирование, ограничение частоты запросов).

Использование gateway позволяет скрыть внутреннюю структуру микросервисной системы от клиента и обеспечить единый контракт взаимодействия через HTTP API. Это упрощает развитие платформы, так как изменения внутреннего распределения функциональности между сервисами не требуют изменений со стороны клиентского приложения при сохранении публичного интерфейса gateway.

После обработки входящего HTTP-запроса gateway взаимодействует с внутренними микросервисами по протоколу gRPC. Для решения задач предметной области выделены специализированные сервисы: course (управление курсами и учебными модулями), user (управление учетными записями и профилями пользователей), auth (процедуры аутентификации, получение и проверка токенов), role (управление ролями и правами доступа), file (работа с файлами и учебными материалами), а также другие сервисы, расширяющие функциональность платформы.

Переход на gRPC для межсервисного взаимодействия обусловлен необходимостью высокой производительности и формальной спецификации интерфейсов. gRPC обеспечивает строгую типизацию контрактов за счёт использования IDL (Protocol Buffers), снижает накладные расходы по сравнению с текстовыми форматами и позволяет эффективно реализовывать синхронные вызовы между сервисами. Кроме того, единый формат описания сообщений упрощает сопровождение и согласование изменений при развитии системы.

Gateway выполняет роль оркестратора, формируя последовательность вызовов внутренних сервисов в зависимости от бизнес-сценария. Например, при

запросе на получение информации о курсе gateway может инициировать обращение к сервису course для получения данных о курсе, затем к сервису role для проверки прав доступа пользователя и при необходимости к сервису file для формирования ссылок на учебные материалы. Такой подход обеспечивает централизованный контроль сценариев и снижает связанность клиентского приложения с внутренней логикой распределения функций.

Для обеспечения безопасности межсервисного взаимодействия в системе предусматривается передача контекста пользователя от gateway к микросервисам. В качестве контекста может использоваться идентификатор пользователя и набор утверждений (claims), извлечённых из токена доступа, либо сервисный токен, подтверждающий полномочия gateway. Это позволяет внутренним сервисам выполнять дополнительные проверки доступа и корректно вести аудит действий пользователя.

Таким образом, организация взаимодействия через единый HTTP gateway и использование gRPC между микросервисами обеспечивает структурированность архитектуры, улучшает производительность и упрощает дальнейшее масштабирование платформы. Выделение специализированных сервисов по доменным областям позволяет развивать функциональность независимо, минимизируя влияние изменений одного компонента на остальные части системы.

☐	auth-6d74c958cf-99s4b	default		0.000	8.2MiB	0	Replica'	worker	Burstat	17d	Running
☐	course-54cd88fbb6-h4xq9	default		0.001	10.9MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	course-activating-monitor-5c	default		0.000	4.8MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	course-monitor-6dbb7f597-c	default		0.002	9.8MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	files-7784985fd6-xg6jv	default		0.005	14.9MiB	0	Replica'	worker	Burstat	17d	Running
☐	fluent-bit-b58xn	default		0.000	3.6MiB	0	Daemon'	worker	Guaran	15d	Running
☐	fluent-bit-lkr29	default		0.000	4.5MiB	0	Daemon'	worker	Guaran	15d	Running
☐	fluent-bit-np5qm	default		0.000	3.8MiB	0	Daemon'	worker	Guaran	15d	Running
☐	fluent-bit-rq7g8	default		0.000	4.6MiB	0	Daemon'	worker	Guaran	15d	Running
☐	frontend-67fb88dc4c-5qdc5	default		0.000	8.8MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	gateway-5ffd564bdd-tft79	default		0.000	14.7MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	kafka-ui-dcbf5d74d-dttbq	default		0.001	436.3M	0	Replica'	worker	BestEffi	185d	Running
☐	ory-keto-6bd88d8bb8-687tr	default		0.000	18.0MiB	0	Replica'	worker	Burstat	221d	Running
☐	promo-58d96d9784-8whnz	default		0.000	5.2MiB	0	Replica'	worker	Burstat	89d	Running
☐	role-service-6d4f9ffb8-ff29l	default		0.000	5.4MiB	0	Replica'	worker	Burstat	5d11h	Running
☐	telegram-78798ffcd5-h7qg2	default		0.000	4.8MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	telegram-consumer-b4867d4	default		0.003	7.3MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	telegram-webhook-768d47b1	default		0.000	5.7MiB	0	Replica'	worker	Burstat	2d6h	Running
☐	user-fcd5489f8-xflj6	default		0.000	9.1MiB	0	Replica'	worker	Burstat	9d	Running
☐	video-consumer-54dd98f786	default		0.002	23.5MiB	0	Replica'	worker	Burstat	15d	Running
☐	video-progress-848cf9589c-h	default		0.001	7.0MiB	0	Replica'	worker	Burstat	5d9h	Running
☐	video-progress-consumer-54	default		0.002	7.3MiB	0	Replica'	worker	Burstat	5d9h	Running
☐	video-server-9ddd4c5-dvx2t	default		0.001	22.9MiB	0	Replica'	worker	Burstat	11d	Running

Рис. 14 Запущенные сервисы в Kubernetes

Проектируемая архитектура ориентирована на горизонтальное масштабирование. Серверные компоненты реализуются по принципу отсутствия состояния, что позволяет запускать несколько экземпляров приложения и распределять нагрузку между ними с помощью балансировщика.

Хранилища данных и сервисы аутентификации размещаются с учётом исключения единых точек отказа. Использование объектного хранилища для мультимедийного контента позволяет эффективно обрабатывать большие объёмы информации и снижает нагрузку на серверную часть.

Предусмотренные архитектурные решения обеспечивают устойчивость платформы к росту пользовательской активности и создают основу для дальнейшего функционального развития системы.

3. Технологии хранения данных

Выбор технологий хранения данных является одним из ключевых этапов проектирования образовательной веб-платформы, поскольку от используемых механизмов хранения напрямую зависят производительность системы, надёжность обработки информации, масштабируемость и безопасность пользовательских данных. В рамках разрабатываемого сайта для проведения обучающих курсов система хранения должна обеспечивать работу с различными типами данных, включая учетные записи пользователей, сведения о курсах, результаты обучения и мультимедийный контент.

При формировании архитектуры хранения учитывались следующие требования: обеспечение целостности и согласованности данных, высокая скорость доступа, возможность горизонтального масштабирования, устойчивость к отказам, а также совместимость с микросервисной архитектурой системы.

1. Реляционная база данных

В качестве основной системы хранения структурированных данных выбрана реляционная система управления базами данных PostgreSQL. Данное решение обусловлено высокой надёжностью, поддержкой транзакций, соответствием принципам ACID и широкими возможностями масштабирования.

PostgreSQL используется для хранения информации о пользователях, курсах, ролях, результатах обучения и других сущностях предметной области. Применение реляционной модели позволяет обеспечить строгую структуру данных, поддерживать связи между таблицами и гарантировать целостность информации при выполнении операций изменения.

Дополнительным преимуществом PostgreSQL является поддержка расширенных типов данных, индексации, полнотекстового поиска и механизмов репликации, что делает данную СУБД эффективным решением для образовательных платформ с растущей нагрузкой.



Рис. 15 Логотип PostgreSQL

2. In-memory база данных

Для ускорения операций аутентификации и снижения нагрузки на основную базу данных применяется in-memory хранилище Redis. Данный компонент используется для хранения временных данных, включая токены доступа, сессионную информацию и других целей.

Применение Redis позволяет значительно сократить время обработки запросов, поскольку операции чтения и записи выполняются в оперативной памяти. Это особенно важно для процедур аутентификации, которые выполняются при каждом обращении пользователя к защищённым ресурсам.

Кроме того, Redis поддерживает механизмы автоматического удаления устаревших данных, что удобно для управления временем жизни токенов и сессий пользователей.



Рис. 16 Логотип Redis

3. Объектное хранилище

Образовательные платформы характеризуются значительным объёмом мультимедийных данных, включая видеолекции, презентации и дополнительные материалы. Хранение таких данных в реляционной базе данных является неэффективным, поэтому используется объектное S3-совместимое хранилище.

Объектное хранилище обеспечивает масштабируемость, высокую доступность и возможность эффективной доставки контента пользователям. Файлы хранятся отдельно от основной базы данных, а в PostgreSQL сохраняются только ссылки и метаданные.

Такой подход снижает нагрузку на серверную часть и позволяет обрабатывать большие объёмы учебных материалов без ухудшения производительности системы.

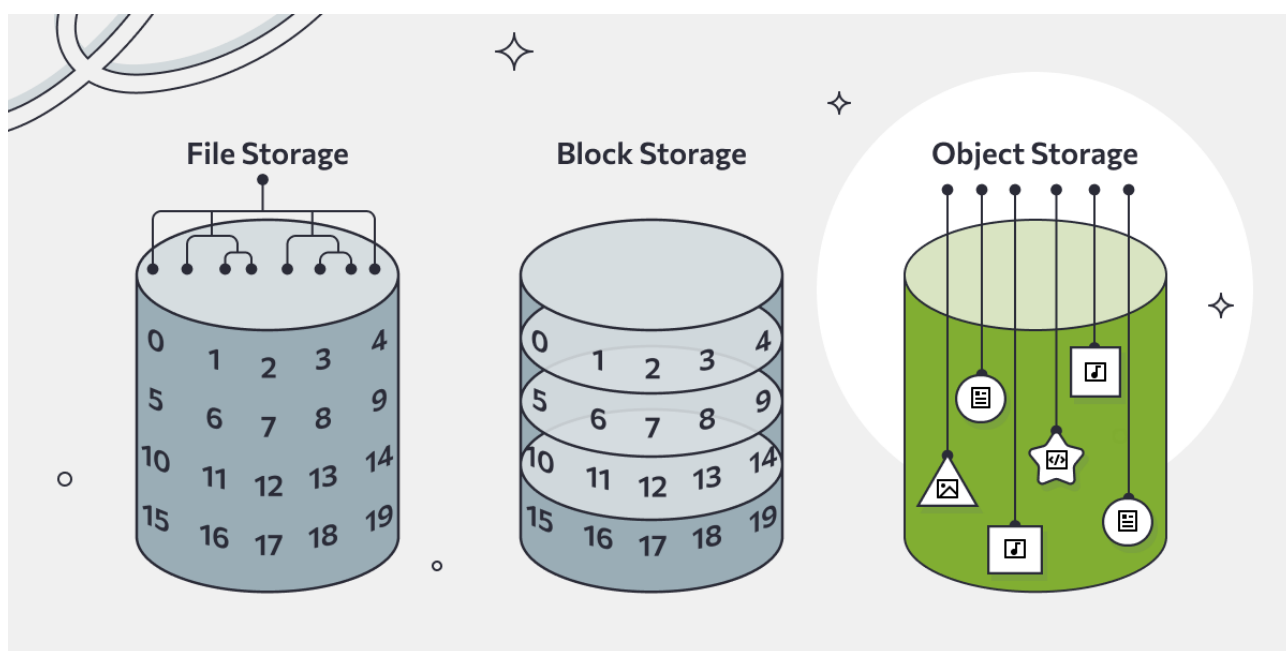


Рис. 17 Отличие объектного хранилища от остальных

4. Пользовательский интерфейс

Реализация пользовательского интерфейса образовательной веб-платформы является завершающим этапом проектирования системы, обеспечивающим непосредственное взаимодействие пользователя с функциональными возможностями сайта проведения обучающих курсов. От качества интерфейсных решений зависят удобство использования платформы, скорость освоения функциональности, а также общая удовлетворённость пользователей образовательным процессом.

Современные образовательные системы предъявляют повышенные требования к интерфейсам, включая адаптивность отображения на различных устройствах, логичность навигации, минимизацию количества действий для выполнения основных сценариев и соответствие требованиям доступности. В связи с этим разработка пользовательского интерфейса выполнялась с учётом принципов человеко-ориентированного проектирования и единообразия визуальных компонентов.

Весь пользовательский интерфейс был полностью спроектирован в программе Figma и реализован с помощью React JS.

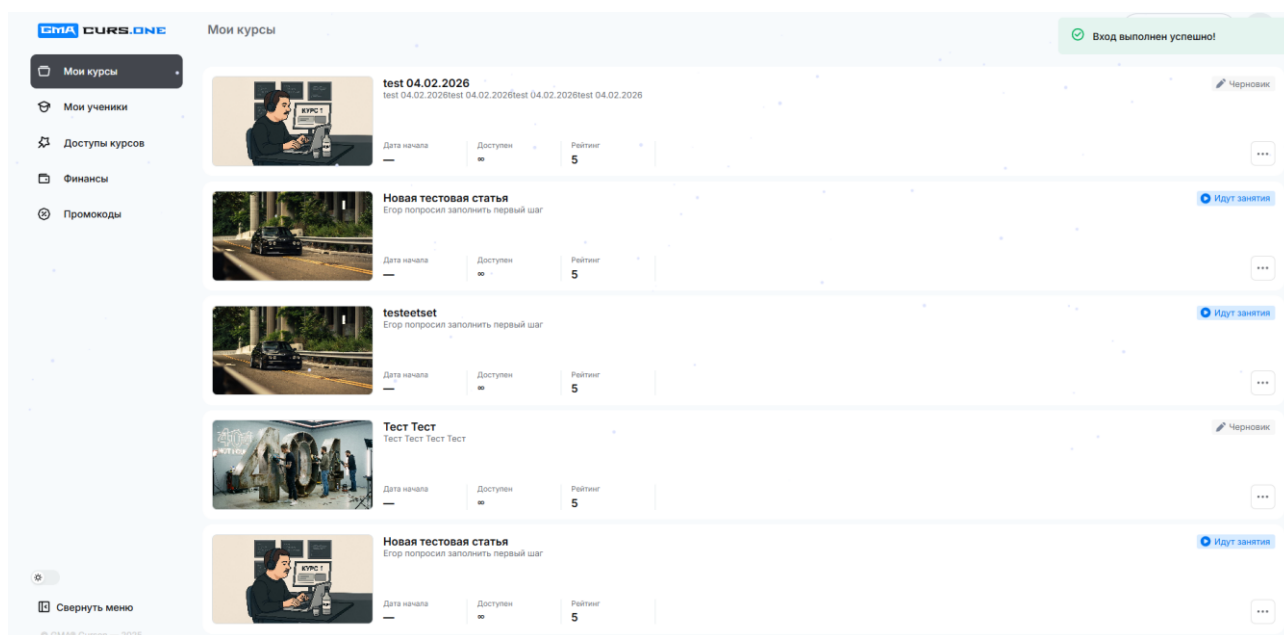


Рис. 18 Пример личного кабинета автора

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы были достигнуты следующие цели:

1. Проведен анализ современных подходов к организации онлайн-обучения и архитектуры образовательных веб-платформ.
2. Исследованы существующие системы аутентификации и авторизации, реализующие механизм единого входа (SSO).
3. Выполнен сравнительный анализ готовых решений и подхода разработки собственной SSO-системы.
4. Спроектирована архитектура сайта проведения обучающих курсов с использованием микросервисного подхода.
5. Выбраны и обоснованы технологии хранения данных.
6. Разработана структура пользовательского интерфейса и описаны основные сценарии взаимодействия пользователя с платформой.

Результаты выполненной курсовой работы показали, что разработка образовательной веб-платформы требует комплексного подхода, учитывающего архитектуру системы, безопасность взаимодействия пользователей и устойчивость инфраструктуры к росту нагрузки. Проведённый анализ подтвердил, что применение микросервисной архитектуры совместно с централизованной системой аутентификации позволяет обеспечить гибкость развития системы и надёжность разграничения доступа к образовательным ресурсам.

Особое внимание в работе было уделено выбору SSO-решения и механизмов авторизации. Сравнение существующих подходов показало, что использование модульной экосистемы ORY обеспечивает оптимальное сочетание безопасности, масштабируемости и независимости от внешних облачных сервисов. Реализация взаимодействия через единый HTTP-gateway и внутренние gRPC-сервисы позволила сформировать структурированную архитектуру, способную эффективно обрабатывать пользовательские запросы и поддерживать дальнейшее расширение функциональности платформы.

Выбранные технологии хранения данных, включающие PostgreSQL, Redis и S3-совместимое объектное хранилище, продемонстрировали целесообразность разделения данных по типам нагрузки и способам доступа. Такой подход обеспечивает высокую скорость обработки запросов, целостность информации и устойчивость системы при увеличении объёма образовательного контента и числа пользователей.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. RFC 6749. «The OAuth 2.0 Authorization Framework» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 05.10.2025 - 06.02.2026);
2. RFC 6750. «The OAuth 2.0 Authorization Framework: Bearer Token Usage» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc6750> (дата обращения: 05.10.2025 - 06.02.2026);
3. RFC 7636. «Proof Key for Code Exchange by OAuth Public Clients (PKCE)» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc7636> (дата обращения: 05.10.2025 - 06.02.2026);
4. RFC 8252. «OAuth 2.0 for Native Apps» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/rfc8252/> (дата обращения: 05.10.2025 - 06.02.2026);
5. RFC 7009. «OAuth 2.0 Token Revocation» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc7009> (дата обращения: 05.10.2025 - 06.02.2026);
6. OpenID Connect Core 1.0 (Errata Set 2). «OpenID Connect Core 1.0» [Электронный ресурс] – URL: https://openid.net/specs/openid-connect-core-1_0.html (дата обращения: 05.10.2025 - 06.02.2026);
7. OpenID Connect Discovery 1.0 (Errata Set 2). «OpenID Connect Discovery 1.0» [Электронный ресурс] – URL: https://openid.net/specs/openid-connect-discovery-1_0.html (дата обращения: 05.10.2025 - 06.02.2026);
8. RFC 7519. «JSON Web Token (JWT)» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/rfc7519/> (дата обращения: 05.10.2025 - 06.02.2026);
9. RFC 7515. «JSON Web Signature (JWS)» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc7515> (дата обращения: 05.10.2025 - 06.02.2026);
10. OWASP. «Application Security Verification Standard (ASVS)» [Электронный ресурс] – URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата обращения: 05.10.2025 - 06.02.2026);
11. OWASP Cheat Sheet Series. «JSON Web Token for Java Cheat Sheet» [Электронный ресурс] – URL: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html (дата обращения: 05.10.2025 - 06.02.2026);
12. NIST. «SP 800-63B: Digital Identity Guidelines: Authentication and Lifecycle Management» [Электронный ресурс] – URL: <https://pages.nist.gov/800-63-3/sp800-63b.html> (дата обращения: 05.10.2025 - 06.02.2026);
13. NIST CSRC. «SP 800-63B (Update 2): Digital Identity Guidelines: Authentication and Lifecycle Management» [Электронный ресурс] – URL:

- <https://csrc.nist.gov/pubs/sp/800/63/b/upd2/final> (дата обращения: 05.10.2025 - 06.02.2026);
14. Ory. «Ory Hydra Documentation (API Reference)» [Электронный ресурс] – URL: <https://www.ory.com/docs/hydra/reference/api> (дата обращения: 05.10.2025 - 06.02.2026);
15. Ory. «Ory Keto (репозиторий проекта)» [Электронный ресурс] – URL: <https://github.com/ory/keto> (дата обращения: 05.10.2025 - 06.02.2026);
16. Keycloak. «Documentation» [Электронный ресурс] – URL: <https://www.keycloak.org/documentation> (дата обращения: 05.10.2025 - 06.02.2026);
17. Auth0. «Auth0 Docs» [Электронный ресурс] – URL: <https://auth0.com/docs> (дата обращения: 05.10.2025 - 06.02.2026);
18. Auth0. «Authentication API» [Электронный ресурс] – URL: <https://auth0.com/docs/api/authentication/> (дата обращения: 05.10.2025 - 06.02.2026);
19. Firebase. «Firebase Authentication Documentation» [Электронный ресурс] – URL: <https://firebase.google.com/docs/auth> (дата обращения: 05.10.2025 - 06.02.2026);
20. OAuth.net. «OAuth 2.0» [Электронный ресурс] – URL: <https://oauth.net/2/> (дата обращения: 05.10.2025 - 06.02.2026);